NanoDiT: Scaling Down Diffusion Transformers for Efficient On-Device Generation

Anonymous CVPR submission

Paper ID

Abstract

Diffusion Transformers (DiT) have established a new state-of-the-art in high-fidelity image synthesis; however, their massive computational complexity and memory requirements hinder local deployment on resource-constrained edge devices. In this paper, we introduce NanoDiT, a family of hardware-efficient generative transformers specifically engineered for mobile Neural Processing Units (NPUs), such as the Qualcomm Hexagon and Apple Neural Engine (ANE). By leveraging a hardware-aware optimization framework, we systematically identify and prune structural redundancies within the DiT backbone that are particularly taxing for mobile data-flows. Our approach yields a series of lightweight models that achieve a 20–30% reduction in parameters and a 25–40% decrease in FLOPs without sacrificing the scaling advantages or the expressive capacity of the original transformer architecture. Extensive benchmarking demonstrates that NanoDiT offers a superior Pareto-optimal trade-off between Fréchet Inception Distance (FID) and inference latency compared to both optimized mobile U-Nets and vanilla DiT variants. By enabling responsive, private, and offline generative AI directly on-device, NanoDiT provides a scalable blueprint for transitioning large-scale foundation models from high-end GPUs to the palm of the user.

1. Introduction

Diffusion models have recently emerged as a dominant paradigm for high-fidelity image synthesis and restoration. Early works such as Denoising Diffusion Probabilistic Models (DDPM) [4] demonstrated that iterative denoising processes can generate high-quality images by gradually transforming Gaussian noise into structured samples. Subsequent developments such as Latent Diffusion Models (LDM) [7] improved scalability by performing diffusion in a compressed latent space, enabling high-resolution generation with significantly reduced computational cost.

More recently, transformer-based diffusion architectures have shown strong scaling behavior for generative modeling. Diffusion Transformers (DiT) [6] replace convolutional U-Net backbones with transformer blocks that operate on image latent tokens, achieving state-of-the-art generative performance on large-scale datasets. However, this improved modeling capability comes at the cost of significant computational complexity and memory consumption. Large variants such as DiT-XL/2 contain hundreds of millions of parameters and require substantial compute and memory bandwidth during inference.

These requirements present a major barrier to deploying diffusion transformers on *edge devices* such as smartphones and embedded AI accelerators. Although cloud-based inference is widely used, on-device generation is increasingly desirable due to benefits such as reduced latency, enhanced privacy, and offline accessibility. Unfortunately, large diffusion transformer models often exceed the strict compute, memory, and thermal constraints of mobile hardware.

To address this challenge, several recent works have proposed lightweight diffusion architectures. MobileDiffusion [10] introduces architectural simplifications and efficient transformer components designed specifically for mobile devices. Other approaches explore pruning and structural redundancy removal to reduce model complexity. Diff-Pruning [2] proposes structured pruning techniques to remove redundant parameters in diffusion models. Similarly, FlexDiT [1] introduces dynamic token density control to reduce unnecessary computation during diffusion inference. Hardware-aware diffusion frameworks such as NanoSD [8] further demonstrate that carefully optimized architectures can significantly improve the efficiency of diffusion models for edge deployment.

Despite these advances, designing efficient diffusion transformers remains challenging. Existing approaches often rely on manual architecture design or heuristic pruning strategies, which may fail to fully exploit the redundancy present in large transformer architectures. A systematic exploration of architectural trade-offs in diffusion transformers—while jointly considering hardware con-

straints—remains relatively underexplored.

In this work, we introduce a hardware-aware architecture optimization framework for scaling down Diffusion Transformers for edge deployment. Starting from the DiT-XL/2 backbone, we construct a large design space of hardware-friendly surrogate architectures by modifying the internal structure of transformer blocks. Our search space includes structural simplifications such as removing transformer blocks, modifying MLP expansion ratios, and reducing hidden dimensional ranks within transformer layers.

To efficiently train surrogate architectures, we employ layer-wise knowledge distillation [3], where each surrogate block learns to mimic the behavior of its corresponding block in the teacher model. This strategy enables rapid training of candidate architectures while preserving the functional characteristics of the original model.

We further formulate architecture selection as a multi-objective Bayesian optimization problem [9] that jointly considers generative performance and hardware efficiency metrics such as peak memory usage. This framework allows us to identify Pareto-optimal architectures that balance generation quality and hardware efficiency.

Through this approach, we obtain a family of compact diffusion transformer architectures that significantly reduce computational cost while maintaining strong generative performance, enabling practical deployment of diffusion transformers on resource-constrained edge devices.

Contributions.

- We propose a hardware-aware architecture optimization framework for compressing Diffusion Transformers starting from the DiT-XL/2 backbone.
- We design a two-stage surrogate architecture generation strategy that includes transformer block removal and structural simplifications through modified MLP expansion ratios and reduced hidden dimensional ranks.
- We employ layer-wise knowledge distillation to efficiently train surrogate architectures while preserving the functional behavior of the original model.
- We use multi-objective Bayesian optimization to identify Pareto-optimal architectures that jointly optimize generative performance and hardware efficiency.

2. Related Work

2.1. Diffusion Models

Diffusion models haveemerged as one of the most powerful generative modeling frameworks in recent years. Denoising Diffusion Probabilistic Models (DDPM) [4] introduced a probabilistic formulation for image generation through iterative denoising of Gaussian noise. Subsequent work such as Improved DDPM [5] improved training stability and sampling efficiency. Latent Diffusion Models (LDM) [7] further improved scalability by performing diffusion in a compressed latent space, enabling high-resolution generation while reducing computational requirements.

2.2. Diffusion Transformers

While early diffusion models relied primarily on convolutional U-Net architectures, recent work has explored transformer-based diffusion backbones. Diffusion Transformers (DiT) [6] treat image latents as sequences of tokens and apply transformer blocks to model long-range dependencies in the latent representation. This architecture exhibits strong scaling properties and achieves state-of-the-art performance on large-scale generative tasks. However, the large computational and memory requirements of transformer-based diffusion models make them difficult to deploy in resource-constrained environments.

2.3. Efficient and Lightweight Diffusion Models

Given the computational cost of modern diffusion architectures, several works have explored designing lightweight diffusion models. MobileDiffusion [10] proposes architecture modifications and efficient transformer components to enable practical diffusion inference on mobile devices. Diff-Pruning [2] introduces structured pruning strategies that remove redundant parameters and layers in diffusion networks to reduce model size and computational cost. FlexDiT [1] further improves efficiency by introducing dynamic token density control to reduce unnecessary computation in diffusion transformers. These approaches highlight the substantial redundancy present in large diffusion architectures and motivate the development of more efficient diffusion transformer variants.

2.4. Model Compression and Architecture Search

Model compression techniques such as pruning, knowledge distillation, and neural architecture search have been widely used to obtain compact neural networks suitable for edge deployment. Knowledge distillation [3] enables smaller student models to learn the behavior of larger teacher networks and has proven effective across various deep learning tasks.

Automated architecture search methods such as Bayesian optimization [9] have also been used to discover efficient neural architectures under hardware constraints. In the context of diffusion models, hardware-aware approaches such as NanoSD [8] demonstrate that carefully optimized architectures can significantly improve efficiency for real-time edge deployment.

In contrast to prior work that primarily focuses on manual architectural design or pruning, our approach introduces a surrogate-based architecture exploration framework combined with layer-wise distillation and multi-objective Bayesian optimization to systematically discover efficient

diffusion transformer architectures suitable for edge deployment.

3. Method

Our goal is to design edge-efficient variants of Diffusion Transformers from DiT-XL/2 [6] baseline, that preserve generative performance while reducing the computational and memory overhead for edge deployment. Moreover, theoritical compute metrics (FLOPs/GMACs) may not always reliably predict pratical latency, since NPUs are optmized for specific operations such as GEMM. So, reducing the arithmetic compute may not always gaurantee propotional edge latency improvements. To address this bridge, we construct a structured architecture search space of lightweight surrogate transformer blocks, which are in align with edge hardware. These surrogates are trained using feature-wise knowledge distillation and these locally distilled surrogates are combined to form candidate architectures, which are then fully trained End to End for accuracy improvements. We employ multi-objective bayesian optimization to identify these pareto-optimal candidate models in the hadware-performance search space. From this set, we select the most balanced architecture and fix it as our base architecture *NanoDiT*. An overview of the proposed framework is illustrated in Fig. X.

3.1. Hardware Decomposition of Diffusion Transformers

Diffusion Transformers (DiT) [6] operate on latent image representations by treating them as sequences of tokens processed by transformer blocks. Given an input image x, it is first encoded into a latent representation using a variational autoencoder. The latent representation is then divided into non-overlapping patches of size $p \times p$, producing a sequence of tokens that form the input to the transformer backboneA DiT model consists of a stack of L ($L = 28$ for DiT-XL/2) transformer blocks. Each block contains two primary components:

- Multi-head self-attention (MHSA)
- A feed-forward network (FFN) with an expansion ratio r

Formally, a transformer block can be expressed as

$$h_{l+1} = h_l + \text{MHSA}(\text{LN}(h_l)) + \text{FFN}(\text{LN}(h_l)), \quad (1)$$

where h_l denotes the hidden representation at layer l and LN denotes layer normalization.

While DiT models provide strong generative performance, their large number of layers and high-dimensional hidden representations introduce significant computational and memory overhead. To address this challenge, we decompose the DiT architecture into smaller hardware-friendly building blocks and construct a structured architecture search space.

Our hardware-aware search space consists of the below surrogates.

1. **Block removal**: Every two consecutive DiT layers are chosen to remove to reduce the network depth.
2. **MLP ratio modification**: The expansion ratio of the FFN block in DiT layers is varied (expansion factor $= r \in \{1, 2, 3, 4\}$) to reduce parameter count and FLOPs.
3. **Hidden dimension reduction**: The internal projection dimensions ($d \in \{512, 1152\}$) of the transformer blocks in the DiT layers are reduced to construct low-rank representations.
4. Refer to Figures 1a, 1b, 1c respectively for the surrogates list.

The above three types of surrogates are built in two stages.

- **Stage 1** consists of choosing among the options in Block removal, which results in replacing two consecutive DiT layers with a single DiT layer, resulting in a search space of $(2 \times 2 \times ...(14 \text{ times}) \times 2 = 2^{14})$
- **Stage 2** consists of choosing either the MLP modified layer or the Hidden dimension reduced layer, so per each layer we have $4 + 2 = 6$ options, resulting in a search space of $(6 \times 6 \times ...(28 \text{ times}) \times 6 = 6^{28})$

These modifications gave a combined a hardware-aware search space of $\mathbf{2^{14} + 6^{28}}$. All the possible options of surrogates are listed in the Figure 1

3.2. Feature-wise Knowledge Distillation

Training every candidate architecture in $2^{14} + 6^{28}$ from scratch would be computationally expensive and infeasible. To enable efficient training, we adopt a feature-wise knowledge distillation strategy, which is a divide-and-conquer strategy, where each surrogate block learns to mimic the behavior of the corresponding block in the teacher network.

Let $T_l(x)$ denote the output feature of the l-th block in the teacher model and $S_l(x)$ denote the output of the corresponding surrogate block for a given input features x. The feature-wise distillation loss is defined as

$$\mathcal{L}^{\text{l}}_{\text{KD}} = ||T_l(x) - S_l(x)||_2^2, \quad (2)$$

We perform surrogate training in two stages as mentioned above as well:

Stage 1: Block removal. Each pair of two consecutive DiT layers are treated as teacher model and their info is distilled into single student DiT layer.

Stage 2: Structural Simplification. In the second stage, the DiT layers acts as teacher blocks and DiT layers with varying MLP ratio, reduced hidden dimensions acts as students.

Since all blocks are distilled independently, this process in massively parallel and computationally lighweight. We obtain 14 surrogates in stage 1, and $5 \times 28 = 140$ surrogates in stage 2.

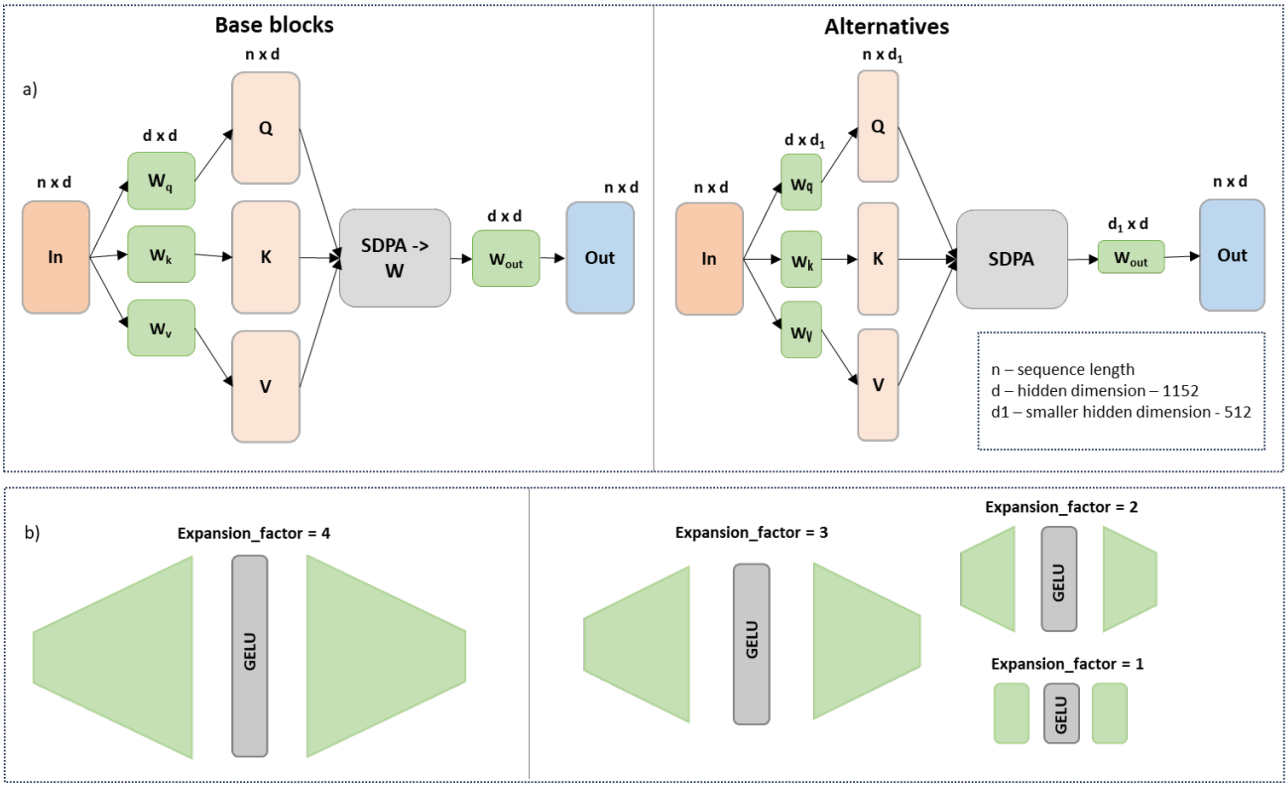

Figure 1. hardware-aware surrogates search space design

This progressive training strategy allows us to efficiently learn light weight surrogate components, preserving DiT-XL/2's local behavior. Which enables to get the full candidate architecture by just by assembling these surrogates to approximate the DiT-XL/2 without full architecture training.

3.3. Assembling Candidate Architectures

Once the surrogate blocks are trained, we construct candidate diffusion transformer architectures by combining different configurations of surrogate components.

Each candidate architecture is represented by a configuration vector

$$a = (b_1, b_2, \ldots, b_L), \tag{3}$$

where b_i denotes the block type selected for layer i. The block type can correspond to the original DiT block or one of the surrogate variants.

Given a configuration a, the corresponding network is assembled by selecting the appropriate surrogate blocks for each layer. Because surrogate blocks are pre-trained through distillation, candidate architectures can be evaluated efficiently without requiring exhaustive retraining.

3.4. Optimization in Hardware-Aligned Space

To identify efficient architectures within the search space, we formulate model selection as a multi-objective optimization problem. Specifically, we aim to simultaneously optimize generative performance and hardware efficiency.

Let $f(a)$ denote the performance metric (e.g., FID or Inception Score) of architecture a, and let $g(a)$ denote a hardware cost metric such as peak memory usage or edge latency. Our objective is to identify architectures that lie on the Pareto frontier, for any architecture a we aim to:

$$\max_{a \in \mathcal{A}} f(a) \quad \text{and} \quad \min_{a \in \mathcal{A}} g(a) \tag{4}$$

To efficiently explore the architecture space, we employ multi-objective-bayesian optimization [9] (MOBO). A gaussian model is trained to predict the objective values of candidate architectures based on previously evaluated configurations. New architectures are then selected using an acquisition function that balances exploration and exploitation.

This process enables efficient discovery of architectures that achieve favorable trade-offs between generative quality and hardware efficiency. For our experiments, we construct some set of images from each of candidate architecture and we took $f(a)$ as the FID score and $g(a)$ as the edge latency

of the particular candidate architecture.

Since the FID caluclation process also remains costly even without training, we adapt MOBO where we relax a to a continous representation $x \in [0,1]^{28}$ and map it to the nearest feasible architecture via a determinstic mapping. candidate architectures are selected by maximizing Expected Hypervolume Improvement (EHVI).

3.5. End-to-End Training

After identifying Pareto-optimal candidate architectures from the search process, the selected models are trained end-to-end to fully adapt the network parameters. During this stage, all model components are jointly optimized using the standard diffusion training objective.

Given a noisy latent representation z_t at timestep t, the model predicts the corresponding noise $\epsilon_\theta(z_t, t)$. The diffusion training loss is defined as

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0,\epsilon,t}\left[||\epsilon - \epsilon_\theta(z_t, t)||_2^2\right], \tag{5}$$

where z_0 is the clean latent representation and ϵ is Gaussian noise.

This final training stage allows the selected architecture to fully adapt to the diffusion task, resulting in compact models that maintain strong generative performance while satisfying hardware constraints.

4. Experimental Results

4.1. Pareto Analysis and Model Selection

4.2. Server Benchmarking

we have caluclated the Parameters, GFLOPs, GMAcs of the DiT family of models (S, B, L, XL) and our NanoDiT for both (256,256) and (512,512) image generation. Results have been reported in table 1

4.3. Edge Device Benchmarking

We have profilled the DiT family of models (S, B, L, XL) and our NanoDiT on Apple A18 Pro ANE (Iphone 16 Pro Max) and Qualcomm 8850 NPU (Samsung Galaxy S25 Ultra). Results have been reported in Table 2 .

5. Conclusion

Nano DiT demonstrates that the scalability of Diffusion Transformers is not inherently at odds with edge deployment. By leveraging layer-wise distillation and Bayesian-optimized architecture search, we successfully compressed the state-of-the-art DiT-XL/2 by 30%. Our approach provides a robust framework for bringing large-scale generative models to mobile NPUs, ensuring privacy, speed, and efficiency for the next generation of mobile AI.

References

[1] Shuning Chang, Pichao Wang, Jiasheng Tang, and Yi Yang. Flexdit: Dynamic token density control for diffusion transformers. *arXiv preprint arXiv:2412.06028*, 2024. 1, 2

[2] Gongfan Fang et al. Diff-pruning: Structural pruning for diffusion models. *arXiv preprint arXiv:2305.10924*, 2023. 1, 2

[3] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 2

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020. 1, 2

[5] Alex Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021. 2

[6] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, 2023. 1, 2, 3

[7] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022. 1, 2

[8] Subhajit Sanyal, Srinivas Soumitri Miriyala, Akshay Bankar, Sravanth Kodavanti, et al. Nanosd: Edge efficient foundation model for real-time image restoration. *arXiv preprint arXiv:2601.09823*, 2026. 1, 2

[9] Jasper Snoek, Hugo Larochelle, and Ryan Adams. Practical bayesian optimization of machine learning algorithms. In *NeurIPS*, 2012. 2, 4

[10] Jiasheng Tang et al. Mobilediffusion: Efficient diffusion models for mobile devices. *arXiv preprint*, 2024. 1, 2

Table 1. Complexity Analysis: Parameters, GMACs, and GFLOPS for varying image resolutions.

Model	Image Size: $1 \times 4 \times 256 \times 256$			Image Size: $1 \times 4 \times 512 \times 512$		
	Params (M)	GMacs	GFlops	Params (M)	GMacs	GFlops
DiT S/2	32.96	6.06	12.13	33.26	31.44	62.93
Dit B/2	130.51	23.01	46.04	131.10	106.38	212.88
DiT L/2	458.10	80.70	161.47	458.89	360.98	722.27
DiT XL /2	675.13	118.62	237.34	676.01	524.54	1050.00
NanoDiT 1	470.65	71.94	143.96	470.76	311.29	560.01
NanoDiT 2	529.64	84.94	169.97	529.69	372.86	662.83

Table 2. Inference Latency (ms) comparison on mobile devices across different resolutions.

Model	Image Size: $1 \times 4 \times 256 \times 256$		Image Size: $1 \times 4 \times 512 \times 512$	
	Iphone Latency	Samsung Latency	Iphone Latency	Samsung Latency
DiT S/2	6.63	8.35	93.31	52
Dit B/2	19.55	24.45	286.58	115.95
DiT L/2	68.99	85.00	790.26	376.08
DiT XL /2	118.56	129.00	1098.92	553.13
NanoDiT 1	70.86	86.13	570.39	258.67
NanoDiT 2	72.53	89.22	688.48	330.27